

Dimensionality Reduction and Clustering in Python

Practical advice for business analytics students

OpenAI

March 17, 2026

1. Start with the data, not the algorithm

Recommended workflow

- ▶ Define the business question first: segmentation, anomaly discovery, visualization, or compression.
- ▶ Separate **numeric**, **categorical**, and **ID/date** fields before modeling.
- ▶ Handle **missing values** before PCA or clustering.
- ▶ Use **scaling** for distance-based methods: PCA, K-means, Gaussian mixtures, and DBSCAN.
- ▶ Investigate **outliers**: extreme values can distort centroids, covariance estimates, and distance thresholds.

Useful preprocessing choices

- ▶ Missing numeric values: median imputation
- ▶ Missing categorical values: most frequent / separate category
- ▶ Scaling: `StandardScaler` is the default starting point
- ▶ Heavy tails: consider `RobustScaler`
- ▶ Outliers: inspect, winsorize, cap, or remove only with business justification

Rule of thumb: If Euclidean distance matters, preprocessing matters even more.

2. Build a reproducible sklearn pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

prep = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

X_prep = prep.fit_transform(X_num)
```

- ▶ **Why pipelines?** They prevent train/test leakage and make your work reproducible.
- ▶ **Missing values first, scaling second.** Scaling with NaNs still present usually breaks the workflow.
- ▶ **Outliers:** if a few rows are clearly erroneous, remove them before fitting; otherwise compare results with and without them.
- ▶ Save the pipeline with your clustering model so new data is transformed in exactly the same way.

3. PCA: reduce dimensions, denoise, and visualize

When PCA helps

- ▶ Many correlated numeric features
- ▶ Need 2D/3D visualization of customers, stores, or products
- ▶ Want faster downstream clustering
- ▶ Need to reduce noise before modeling

Practical advice

- ▶ Standardize first.
- ▶ Choose components using explained variance.
- ▶ Interpret components with care.
- ▶ Less helpful for non-linear cluster shapes.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2,
          random_state=42)
X_pca = pca.fit_transform(X_prep)

print(pca.n_components_)
print(pca.explained_variance_ratio_)
```

Business use: compress 30 KPIs into a smaller set of latent drivers before segmentation.

4. Which clustering method should you try?

Method	Best for	Watch out for	Practical tip
K-means	Compact, roughly spherical clusters	Sensitive to scale and outliers; must choose k	Run several values of k and compare silhouette score plus business meaning
Gaussian Mixture	Soft membership; overlapping segments	Covariance estimates can become unstable with small samples or noise	Useful when one customer can plausibly belong to multiple segments
DBSCAN	Irregular shapes and noise detection	Sensitive to eps and feature scale; struggles in very high dimensions	Often works better after PCA and with domain-guided tuning

Important: There is no single best clustering; choose the option that is statistically sensible and actionable.

5. K-means and Gaussian mixtures in sklearn

K-means

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

km = KMeans(n_clusters=4,
            n_init=20,
            random_state=42)
labels = km.fit_predict(X_pca)
print(silhouette_score(X_pca, labels))
```

- ▶ Start with PCA output if many features are correlated.
- ▶ Profile clusters using original business variables, not only PCA scores.

Gaussian Mixture

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(
    n_components=4,
    covariance_type="full",
    random_state=42)

gmm_labels = gmm.fit_predict(X_pca)
proba = gmm.predict_proba(X_pca)
```

- ▶ Use posterior probabilities when segment membership is fuzzy.
- ▶ Compare models with BIC/AIC, not only visual intuition.

6. DBSCAN, outliers, and evaluation checklist

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.7, min_samples=10)
db_labels = db.fit_predict(X_pca)

# Label -1 means noise / outlier
```

DBSCAN tips

- ▶ Standardize first.
- ▶ Tune eps with a k-distance plot.
- ▶ Expect some observations to be labeled -1.
- ▶ Prefer lower-dimensional representations if the original space is very wide.

Final checklist before presenting results

- ▶ Did you handle missing values consistently?
- ▶ Did you scale the variables used in distance calculations?
- ▶ Did you test sensitivity to outliers and hyperparameters?
- ▶ Can you explain each cluster in plain business language?
- ▶ Did you validate usefulness with stakeholders, not just metrics?

Best practice: Use clustering as a decision-support tool, not as proof that segments "truly exist."